

Conversational SHL Assessment Recommender

Approach Document | AI Intern Take-home

Problem framing

The task is a grounded, multi-turn recommendation problem where the agent must decide *per turn* whether to clarify, recommend, refine, compare, or refuse — over a non-deterministic conversation, with a hard schema, an 8-turn cap, and a 30s/call budget. I treated it as a two-stage pipeline (route, then execute) so the LLM handles ambiguous dialogue while deterministic Python owns every hard guarantee.

Catalog & data

A defensive scraper (`scrape_catalog.py`) walks only the Individual Test Solutions table (`type=1`), follows pagination, and enriches each product detail page for description, test-type letters (A/B/C/D/E/K/P/S), job levels and length. Every selector has a fallback because SHL markup shifts and “happy-path only” is a named failure mode. A verified seed catalog of real SHL products (correct slugs/URLs) ships alongside so the service boots and stays schema-valid even if a scrape is throttled — the scraper overwrites it when it succeeds.

Retrieval

Hybrid, in-process (no external vector DB — the catalog is small enough that a single process stays well inside the latency budget and simplifies deployment):

- **Dense:** MiniLM (all-MiniLM-L6-v2) cosine over a cached matrix — captures paraphrased intent (“works with stakeholders” → personality).
- **Sparse:** a compact BM25 (no dependency) — captures exact names and acronyms (OPQ32r, GSA, Java 8) that embeddings blur.
- **Structured boosts:** additive, capped nudges for requested test-type and job level.
- **Type-coverage guarantee:** if the user explicitly asked for a test type, at least one item of that type is forced into the candidate set — pure ranking otherwise drops a whole requested category and tanks Recall@10 on multi-type needs.

Scores are min-max normalised per query then fused (0.6 semantic / 0.4 lexical, tuned on the dev traces). If embeddings can't load, retrieval degrades to BM25-only rather than crashing.

Agent & prompt design

Stage 1 — Router (one strict-JSON LLM call): given the full history plus the catalog's controlled vocabulary, it accumulates constraints across the *whole* conversation and picks one action. The prompt encodes explicit action rules (never recommend on a vague turn-1; *refine* carries prior constraints forward instead of restarting; injection/off-topic → refuse) and a fixed intent→test-type map so behaviour is consistent.

Stage 2 — Execute (deterministic, plus a grounded LLM pass): for recommend/refine the model picks and orders *only* from the retrieved candidate indices; names and URLs are taken from the catalog object, never the model — a hallucinated URL is structurally impossible. Compare answers are written strictly from the named entries' catalog descriptions. The service is stateless: working state is rebuilt from message history every call.

Hard guarantees (in code, not prompt)

- Schema always valid — coerced and Pydantic-validated on the way out, even on unexpected errors (graceful reply beats a 500).
- Recommendations are always 0 or 1–10 real catalog rows.
- Turn-cap: at the last allowed turn the agent commits to a best-effort shortlist instead of asking again — 8 turns never exceeded.
- Every external dependency degrades: no LLM key → safe deterministic path; LLM returns junk → JSON repair + fallbacks.

Evaluation

A local replay harness (`eval/run_eval.py`) mirrors SHL's grading: a rule-based simulated user answers from persona facts, says "no preference" otherwise, and ends on a shortlist. It scores hard-eval schema validity, Mean Recall@10, and behaviour probes (refuse off-topic/injection, no-recommend-turn-1, compare-grounded). A second suite (`tests/test_agent_pipeline.py`) injects a deterministic fake LLM to prove the full state machine in isolation — 7/7 passing (clarify, recommend, refine-honours-edit, compare, refuse ×2, turn-cap).

What didn't work / how I measured improvement

- **Pure semantic ranking** dropped requested test-type categories entirely; the *refine*→*adds personality* probe failed. Adding the type-coverage guarantee fixed it (probe FAIL→PASS) without hurting other traces.
- **LLM retry/backoff on an unset key** blew the turn timeout in degraded mode; switching to fail-fast when unconfigured kept every path inside budget.
- **Schema drift risk**: early on the model occasionally returned >10 or malformed recs; moving all schema enforcement into Python (not the prompt) made hard-evals pass on 100% of turns even with the LLM and embeddings fully offline.

Stack justification & AI-tool usage

FastAPI + Pydantic (strict schema, async, trivial free-tier deploy); Groq Llama-3.3-70B (fast, generous free tier, OpenAI-compatible) with a one-env-var swap to OpenAI/Gemini; sentence-transformers + a hand-written BM25 (no heavyweight vector DB for a small catalog). AI assistance was used for scaffolding, the scraper's defensive selectors, and prose polish; all design decisions, the routing contract, the grounding/anti-hallucination mechanism, the type-coverage fix, and the evaluation methodology are my own and are defended above.